Q1. What is the purpose of access specifiers in a class? List the three types.

Answer:

Access specifiers in a class control the visibility and accessibility of class members (data members and member functions) from outside the class. They help in implementing data hiding, security and modularity in C++.

Purpose:

- To restrict direct access to certain members of the class.
- To protect data from unintended modification.
- To provide better control over how data is accessed and used.
- To implement the concept of encapsulation.

Three types of access specifiers:

① public : Members declared as public are accessible from anywhere (inside the class, outside the class, and by derived classes).

② private : Members declared as private are accessible only within the class. They cannot be accessed directly from outside the class or by derived classes.

③ protected : Members declared as protected are accessible within the class and by derived classes, but not by other objects or functions.

Q2. Write C++ Program to demonstrate Static variable and static function.

Ans:

```
// Program to demonstrate static variable and static function
#include <iostream>
using namespace std;

class Demo
{
  private:
      static int count;   // static variable
  public:
      Demo()
      {
          count++;        // increment static variable
      }
      static void displayCount()     // static function
      {
          cout << "Total objects created = " << count << endl;
      }
};

int Demo::count = 0;   // initialize static variable

int main()
{
    Demo d1;      // constructor called → count = 1
    Demo d2;      // constructor called → count = 2
    Demo d3;      // constructor called → count = 3

    Demo::displayCount();     // call static function using class name

    return 0;
}
```

Output :

```
Total objects
  created = 3
```

Explanation:

- count is a static variable. It is shared by all objects of the class.
- It gets memory only once and retains its value.
- displayCount() is a static function. It can be called without creating any object using class name and scope resolution operator (::).

Q3. Explain the difference between friend function and member function.

Ans: Friend function and member function are both used in C++ to access data of a class, but they differ in several aspects as shown below:

Friend Function	Member Function
1. A friend function is not a member of the class but it can access private and protected members of the class.	1. A member function is a part of the class and it can access all members (public, private and protected) directly.
2. It is declared using the keyword 'friend' inside the class.	2. It is declared inside the class without using any special keyword.
3. It is called like a normal function using object as an argument.	3. It is called using the object and the dot operator (.).
4. It does not have 'this' pointer.	4. It has an implicit 'this' pointer which points to the current object.
5. It cannot be called using object directly.	5. It can be called using object directly.
6. It is not in the scope of the class.	6. It is in the scope of the class.
7. It is usually used when a non-member function needs access to private data of the class.	7. It is used to perform operations related to the data members of the class.

Example:

```
// Friend Function
#include <iostream>
using namespace std;
class Box
{
   int length;
   friend void showArea(Box b); // friend function declaration
public:
   void setData(int l)
   {
      length = l;
   }
};
void showArea(Box b)      // friend function definition
{
   cout << "Area = " << (b.length * b.length) << endl;
}
```

```
// Member Function
class Box2
{
   int length;
public:
   void setData(int l)
   {
      length = l;
   }
   int area()      // member function
   {
      return length * length;
   }
};
// Calling
Box2 b2;
b2.setData(5);
cout << "Area = " << b2.area() << endl;
```

Thus, friend function is a non-member function that has special access to class members, whereas member function is a part of the class and works on its objects.

Q4. What are inline functions? State two rules for writing inline functions.

Ans: Inline functions are functions that are expanded (or replaced) at the point of function call by the compiler instead of being called in the normal way. This avoids the overhead of function call (such as saving return address, jumping to the function, etc.) and makes the program faster for small functions.

Syntax :

```
inline return_type function_name(parameters)
{
    // function body
}
```

Two rules for writing inline functions :

① The function should be small, i.e., it should contain only a few statements (preferably one line).

② The function should be frequently called in the program.

Example :

// Inline function to find maximum of two numbers

```
inline int max(int a, int b)
{
    return (a > b) ? a : b;
}

int main()
{
    int x = 10, y = 20, m;
    m = max(x, y);    // function inlined at this point
    cout << "Maximum = " << m;
    return 0;
}
```

Explanation :

When max(x, y) is called in main(), the compiler replaces the call with the body of max(), i.e.,

```
m = (x > y) ? x : y;
```

instead of performing a normal function call. Thus it saves time and memory.

Note : The inline keyword is only a request to the compiler. The compiler may ignore it and treat the function as a normal function.

Q5. Write a C++ code snippet to demonstrate an object passed as a parameter to a function.

Ans: In C++, an object can be passed to a function either by value or by reference. Here, we pass an object by reference to avoid copying and to allow modifications.

Program :

```
#include <iostream>
using namespace std;

class Student
{
    int roll;
    string name;
public:
    Student(int r, string n)
    {
        roll = r;
        name = n;
    }
    void display() const
    {
        cout << "Roll: " << roll << ", Name: " << name << endl;
    }
    // Friend function declaration
    friend void updateName(Student &s, string newName);
};

// Function that accepts an object as a parameter
void updateName(Student &s, string newName)
{
    s.name = newName;   // modifies the object passed
}
int main()
{
    Student s1(101, "Anu");
    cout << "Before update: ";
    s1.display();
    updateName(s1, "Anushka");    // object passed to function
    cout << "After update: ";
    s1.display();
    return 0;
}
```

Explanation :

→ The function updateName() takes a Student object as a reference parameter.

→ Any changes made to the object inside the function reflect in the original object.

→ Here, we change the name of the student using the function.

Output :

```
Before update: Roll: 101, Name: Anu
After update: Roll: 101, Name: Anushka
```

Thus, this program demonstrates how an object is passed as a parameter to a function and how the function can access and modify the object's data.

Q6. Create a class Rectangle with a constructor that takes two parameters and displays area.

Ans: We will create a class Rectangle with private data members length and breadth. A parameterized constructor will initialize these members. A member function will calculate and display the area of the rectangle.

Program :

```
#include <iostream>
using namespace std;
class Rectangle
{
  private:
    double length, breadth;          // data members
  public:
    Rectangle(double l, double b)      // parameterized constructor
    {
        length = l;                    // initialize length
        breadth = b;                   // initialize breadth
    }
    void displayArea()                 // member function to display area
    {
        double area = length * breadth;      // calculate area
        cout << "Area of Rectangle = " << area << " square units" << endl;
    }
};
int main()
{
    double l, b;
    cout << "Enter length of rectangle : ";
    cin >> l;
    cout << "Enter breadth of rectangle : ";
    cin >> b;

    Rectangle r(l, b);     // constructor called with two parameters
    r.displayArea();       // call member function
    return 0;
}
```

Rectangle.

length → double

breadth → double

Sample Output :

```
Enter length of rectangle : 12
Enter breadth of rectangle : 5
Area of Rectangle = 60 square units
```

Explanation :

- The class Rectangle has two private data members length and breadth.
- The constructor Rectangle(double l, double b) initializes these members using the values passed as arguments.
- The function displayArea() calculates the area (length × breadth) and displays it.
- In main(), an object r is created by passing length and breadth. Then the displayArea() function is called using this object.

Q7. What is Constructor and write the rules for declaring a constructor.

Ans: A constructor is a special member function of a class which is automatically invoked whenever an object of the class is created. Its main purpose is to initialize the data members of the class.

Syntax :

```
class  ClassName
{
   public:
        ClassName(parameter_list)
        {
             // initialization code
        }
};
```

→ Constructor name is same as the class name.

Rules for declaring a constructor :

1. The constructor name must be same as the class name.
2. A constructor does not have any return type, not even void.
3. A constructor is automatically called when an object is created.
4. Constructors cannot be inherited, virtual or static.
5. Constructors can be overloaded (i.e., more than one constructor with different parameters can exist in a class).
6. If no constructor is defined by the programmer, the compiler provides a default constructor (without parameters).
7. Constructors can have default arguments.

Example :

```
#include <iostream>
using namespace std;
class Demo
{
    int x, y;
 public:
    // constructor
     Demo(int a, int b)
     {
         x = a;
         y = b;
     }
     void display()
     {
         cout << "x = " << x << ", y = " << y << endl;
     }
};
int main()
{
    Demo obj(10, 20);   // constructor is called
    obj.display();
    return 0;
}
```

Output :

```
x = 10, y = 20
```

Explanation :

When object obj is created, constructor Demo(int, int) is automatically called and initializes x and y.

Q8. Compare and contrast a copy constructor and a constructor with default arguments using examples.

Ans: Both copy constructor and constructor with default arguments are used to initialize objects, but they serve different purposes. The key differences are shown below :

Aspect	Copy Constructor	Constructor with Default Arguments
1. Purpose	Used to create a new object as a copy of an existing object.	Used to allow constructor to be called with no arguments or some arguments by providing default values.
2. When Called	Automatically called when a new object is initialized using another object of the same class.	Called when an object is created and arguments are omitted. Compiler uses default values.
3. Syntax	ClassName(const ClassName &obj) { // copy constructor }	ClassName(type p1 = default1, type p2 = default2, ...) { // default constructor }
4. Parameters	Takes a reference to another object of the same class.	Takes parametors with default values.
5. Use	Used for deep/shallow copy of objects.	Used to provide flexibility while creating objects.

6. Example Call	ClassA a1;
	ClassA a2(a1); // calls copy constructor
		ClassA a1; // uses default values
		ClassA a2(10); // one argument
		ClassA a3(10,20); // two arguments

7. Implicitly Provided by Compiler	Yes, if user does not define.	No. Default arguments must be provided by the programmer.

Example 1 : Copy Constructor

```
#include <iostream>
using namespace std;
class Demo
{
    int x;
public:
    Demo(int a)                 // parameterized constructor
    {   x = a; }
    Demo(const Demo &obj)       // copy constructor
    {
        x = obj.x;
        cout << "Copy Constructor Called\n";
    }
    void display()  { cout << "x = " << x << endl; }
};
int main()
{
    Demo d1(10);        // calls parameterized constructor
    Demo d2(d1);        // calls copy constructor
    d1.display();
    d2.display();
    return 0;
}
```

Output :

```
Copy Constructor Called
x = 10
x = 10
```

Example 2 : Constructor with Default Arguments

```
#include <iostream>
using namespace std;
class Demo
{
    int a, b;
public:
    // constructor with default arguments
    Demo(int x = 0, int y = 0)
    {
        a = x;
        b = y;
    }
    void display()
    {  cout << "a = " << a << ", b = " << b << endl; }
};
int main()
{
    Demo d1;            // uses default values (0,0)
    Demo d2(5);         // a=5, b=0
    Demo d3(5,10);      // a=5, b=10

    d1.display();
    d2.display();
    d3.display();
    return 0;
}
```

Output :

```
a = 0, b = 0
a = 5, b = 0
a = 5, b = 10
```

Summary : Copy constructor creates a new object as a copy of an existing object.

Constructor with default arguments allows flexibility in object creation by providing default values for parameters.

Q9. Write a short C++ program to demonstrate overloading of a member function named display().

Ans: In C++, a member function can be overloaded by defining multiple functions with the same name but different parameter lists (number, type or order of parameters). The compiler selects the appropriate function based on the arguments passed.

Program :

```
#include <iostream>
using namespace std;
class Demo
{
    int a, b;
public:
    Demo(int x, int y)
    {
        a = x;
        b = y;
    }
    // Overloaded display() with no parameters
    void display()
    {
        cout << "Values: " << a << ", " << b << endl;
    }
    // Overloaded display() with one parameter
    void display(int num)
    {
        cout << "Sum of values and " << num << " = "
             << (a + b + num) << endl;
    }
    // Overloaded display() with two parameters
    void display(int x, int y)
    {
        cout << " Product of " << x << " and " << y
             << " = " << (x * y) << endl;
    }
};
int main()
{
    Demo d(10, 20);
    d.display();          // calls display() with no arguments
    d.display(5);         // calls display(int)
    d.display(3, 4);      // calls display(int, int)
    return 0;
}
```

Output :

```
Values: 10, 20
Sum of values and 5 = 35
Product of 3 and 4 = 12
```

Note :

Here, the function name display() is same in all three functions, but the parameter list is different. This is called Function Overloading.

Explanation :

- → d.display() : No arguments are passed, so display() with no parameters is called.
- → d.display(5) : One argument (5) is passed, so display(int) is called.
- → d.display(3,4) : Two arguments (3 and 4) are passed, so display(int, int) is called.

* Thus, this program demonstrates overloading of the member function display().

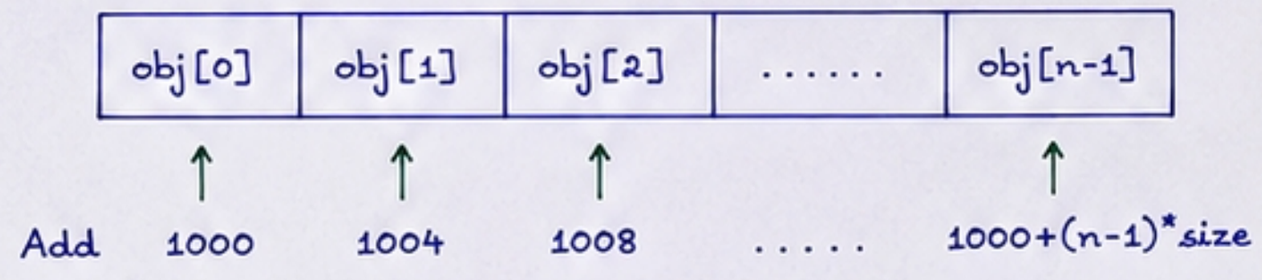
Q10. Explain how array of objects can be created and accessed in C++.

Ans: In C++, an array of objects is a collection of objects of the same class stored in contiguous memory locations. All objects in the array are of the same type. Each object can be accessed using its index just like an array of basic data types.

1. How to create an array of objects :

```
ClassName arrayName[size];
```

2. Memory representation :



3. How to access :

- The objects are accessed using index.
- Syntax : `arrayName[index].memberFunction();`
  or `arrayName[index].dataMember;`

4. Example Program :

```
#include <iostream>
using namespace std;

class Student
{
  public:
      int roll;
      string name;

      void display()
      {
         cout << "Roll: " << roll << ",Name: " << name << endl;
      }
};

int main()
{
    // Array of 3 Student objects
    Student s[3];

    // Input data for each object
    for(int i = 0; i < 3; i++)
    {
       cout << "Enter roll and name of student " << i+1 <<":";
       cin >> s[i].roll >> s[i].name;
    }

    // Access and display each object
    cout << "\nStudent Details:\n";
    for(int i = 0; i < 3; i++)
       s[i].display();

    return 0;
}
```

Output (example) :

```
Enter roll and name of student 1: 101 Ravi
Enter roll and name of student 2: 102 Anu
Enter roll and name of student 3: 103 Sunil

Student Details:
Roll: 101 , Name: Ravi
Roll: 102 , Name: Anu
Roll: 103 , Name: Sunil
```

5. Notes :

* All objects in the array are created when the array is declared.
* Each object can be accessed using index 0 to size - 1.
* We can access both data members and member functions using dot (.) operator.
* Arrays of objects are useful to store records of the same type.

Summary :

An array of objects is created by declaring an array with class type. Each object in the array is accessed using index and the dot operator (.) to access members or call functions.

Q 11. What is a friend function in C++ ? Write a program to demonstrate friend function.

Ans : A friend function in C++ is a function which is not a member of a class but has access to the private and protected members of that class. Friend functions are declared inside the class using the keyword "friend". They are used when a non-member function needs to access or modify private data of the class.

Syntax :

```
class ClassName
{
   private :
         data members;
   public :
         friend return_type function_name(arguments);
};
```

Note :
Friend function is not invoked using object. It is called like a normal function.

Program :

```
#include <iostream>
using namespace std;

class Box
{
    private :
        int length;
        int breadth;
    public :
        Box(int l, int b)
        {
            length = l;
            breadth = b;
        }
        friend int area(Box b);    // friend function declaration
};

// friend function definition
int area(Box b)
{
     return (b.length * b.breadth);    // accessing private members
}

int main()
{
    Box obj(5, 3);
    cout << "Area of box = " << area(obj);  // calling like normal function
    return 0;
}
```

Output :

```
Area of box = 15
```

Explanation :

- The class Box has two private members length and breadth.
- The function area() is not a member of the class but it is declared as friend inside the class.
- Therefore, it can access private members of the class.
- In main(), we create an object and pass it to area() which calculates and returns the area.

Points to Remember :

* Friend function is not in the scope of the class.
* It is declared inside the class but defined outside the class.
* It can access private and protected members of the class.
* It provides a way to share data between classes or to perform operations that require access to private data.



Q 12. Explain the basic structure of a C++ program. Write a simple C++ program to print "Hello, World!".

Ans : A C++ program has a fixed structure which includes a documentation section (comments), preprocessor directives, the main function and statements. Every C++ program is a collection of classes and functions, and the execution of a program begins from the main() function.

Basic Structure :

```
#include <iostream>     // Preprocessor directive
using namespace std;
int main()
{
    // Statements
    return 0;           // Program ends
}
```

Steps in Program Execution :

1. Preprocessor directives are processed.
2. Control transfers to main() function.
3. Statements inside main() are executed sequentially.
4. Control returns to the operating system.

Common C++ Programs :

- Input and output programs
- Arithmetic operations
- Finding largest/smallest number
- Palindrome program
- Factorial program
- Menu driven programs

Example : Hello World Program :

```
# include <iostream>     // Preprocessor directive
using namespace std;
int main()
{
    cout << "Hello, World!" << endl;   // Print message
    return 0;
}
```

Different Operators in C++ :

Type	Operators	Example
Arithmetic	+ , - , * , / , %	a + b , a % b
Relational	== , != , > , < , >= , <=	a == b
Logical	&& , || , !	a && b , !a
Assignment	= , += , -= , *= , /= , %=	a += b
Increment / Decrement	++ , --	a++ , b--
Bitwise	& , | , ^ , ~ , << , >>	a & b , a << 1

Note :
C++ supports structured, procedural and object-oriented programming.

Summary :

* A C++ program starts with preprocessor directives and has a main() function.
* Statements are executed from top to bottom inside main().
* It is a powerful language supporting multiple paradigms.
* Understanding the basic structure helps in writing and solving more complex programs.

Q13. What are tokens in C++ ? Explain the different types of tokens with examples.

Ans : In C++, a token is the smallest individual unit in a program that has a meaning to the compiler. The compiler breaks a C++ program into a series of tokens before processing.
There are five types of tokens in C++.

S.No.	Type of Token	Description	Example
1.	Keywords	These are reserved words in C++ that have special meaning and cannot be used as names.	int, float, if, else, for, while, return, class, public, private, etc.
2.	Identifiers	These are user-defined names given to variables, functions, classes, etc. Rules : Must start with a letter or underscore. Can contain letters, digits and underscores.	sum, totalMarks, _count, number1, displayValue
3.	Literals	These represent fixed values in a program. Types : Integer, Floating point, Character, String, Boolean	25, -10, 3.14, 'A', "Hello", true, false
4.	Operators	These are symbols that perform operations on operands. Types : Arithmetic, Relational, Logical, Assignment, Increment / Decrement, Bitwise, etc.	+, -, *, /, %, ==, !=, <, >, &&, ||, =, ++, --, &, |, ^, etc.
5.	Punctuators	These are symbols used to separate or group other tokens.	;, -, , (,), {, }, [,], :, ::, ?, ->, #, etc.

Example Program :

```
#include <iostream>
using namespace std;
int main()
{
    int a = 10;
    float b = 3.5;
    cout << "Sum = " << (a + b) << endl;
    return 0;
}
```

Token Identification in the above program :

- Keywords → int, float, return, using, namespace
- Identifiers → main, a, b, cout, endl, std
- Literals → 10, 3.5, "Sum = ", 0
- Operators → =, <<, +
- Punctuators → (,), {, }, ;, <, >, ::, .

Notes :

1. Tokens are the basic building blocks of a C++ program.
2. The compiler scans the program from left to right and identifies tokens.
3. Proper use of tokens is essential for writing correct and meaningful programs.

Summary :

C++ tokens are categorized into five types : Keywords, Identifiers, Literals, Operators and Punctuators. They are recognized by the compiler and are essential for the compilation and execution of a program.



Remember

Compiler breaks the program into tokens and processes them.